Лабораторна робота №8

Збереження знань у граф базі даних

Опис:

У цій лабораторній роботі використовується графова база даних (наприклад, Neo4j) для зберігання знань з онтології, побудованої на попередніх етапах. Робота передбачає імпорт знаннь в базу, структурування їх у вигляді графа, а також виконання оптимізованих запитів для отримання потрібної інформації.

Мета роботи:

1. Навчитися використовувати графову базу даних для збереження знань.
2. Розробити запити до бази з метою швидкого та ефективного доступу до інформації.

Інструменти:

- **Neo4j**

Порядок виконання:

1. **Огляд граф баз даних**
   - **Ознайомлення з Neo4j:**
     - Прочитайте документацію щодо використання Neo4j як графової бази даних.
     - Встановіть Neo4j на локальній машині або використовуйте Neo4j Sandbox для онлайн роботи.
   - **Підключення до Neo4j:**
     - Підключіться до Neo4j через браузерний інтерфейс або встановіть Neo4j Desktop для локальної роботи.
     - Переконайтеся, що сервер Neo4j запущений і ви можете взаємодіяти з базою.

2. **Структурування даних**
   - **Перетворення онтології у графову структуру:**
     - Визначте ключові поняття та їх зв'язки з побудованої онтології (наприклад, "Курс", "Студент", "Тема").
     - Структуруйте дані у формат, що підтримується граф базами даних, де кожен вузол відповідає окремому поняттю, а ребра — це взаємозв'язки між ними (наприклад, "Студент проходить Курс").
   - **Підготовка даних до імпорту:**
     - Використайте мову Cypher для опису вузлів та зв'язків між ними.
     - Підготуйте CSV або JSON файли, які містять вузли і зв'язки для імпорту в Neo4j.

Приклад:

```
CREATE (s:Student {name: "John Doe"})
CREATE (c:Course {name: "Introduction to AI"})
CREATE (s)-[:ENROLLED_IN]->(c)
```

3. **Збереження знань у граф базі даних**
   - **Імпорт даних через Cypher або API:**
     - Використайте Cypher для створення вузлів (понять) та встановлення зв'язків між ними. Якщо ви використовуєте API (наприклад, Python з бібліотекою py2neo), створіть скрипт для автоматизованого імпорту.
   - **Перевірка даних:**
     - Перевірте правильність імпортованих даних через інтерфейс Neo4j. Візуалізуйте граф і переконайтеся, що всі вузли і зв'язки відображені коректно.

     Приклад імпорту через Cypher:

     ```
     LOAD CSV WITH HEADERS FROM 'file:///students.csv' AS row
     CREATE (:Student {name: row.name, age: toInteger(row.age)});
     ```

4. **Оптимізація запитів до бази**
   - **Написання запитів для отримання знань:**
     - Напишіть кілька базових запитів, щоб витягнути інформацію з бази. Наприклад, запит про те, які курси проходить конкретний студент.

     ```
     MATCH (s:Student)-[:ENROLLED_IN]->(c:Course)
     WHERE s.name = "John Doe"
     RETURN c.name
     ```

   - **Оптимізація запитів:**
     - Використовуйте індекси для прискорення пошуку даних. Наприклад, створіть індекси для імен студентів або назв курсів.

     ```
     CREATE INDEX ON :Student(name);
     ```

5. **Тестування продуктивності:** Виконайте кілька складних запитів і виміряйте їх продуктивність (час виконання, обсяг даних).

Результати роботи:

- Візуальне відображення бази знань в Neo4j
- Візуальне відображення графових запитів для витягування інформації.